

A Project Report

on

**VOICE COMMAND TEXT CONVERSION**

Submitted in partial fulfilment of requirements for the award of the

Degree of

**BACHELOR OF TECHNOLOGY**

in

**ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**

Under the guidance of

**Mrs. SUBHA SRI S**

**IBM CORPORATE TRAINER**

Submitted by

**ROOBA B**

**[REG NO: 927623BAD092]**

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA  
SCIENCE**

**M KUMARASAMY COLLEGE OF ENGINEERING, KARUR**

(Autonomous)

**Karur 639-113**

**DECEMBER - 2025**

# **M. KUMARASAMY COLLEGE OF ENGINEERING**

**(Autonomous Institution affiliated to Anna University, Chennai)**

**KARUR – 639 113**

## **BONAFIDE CERTIFICATE**

Certified that this project report on **“VOICE COMMAND TEXT CONVERSION”** is the Bonafide work **“ROOBA B (927623BAD092)”** who carried out the project work during the academic year 2025- 2026 under my supervision.

### **SIGNATURE**

**Mrs. SUBHA SRI S**

**IBM CORPORATE TRAINER**

Department of Artificial Intelligence

M. Kumarasamy College of Engineering,

Thalavapalayam, Karur-639 113.

### **SIGNATURE**

**Dr. A. SELVI M.E., Ph.D.,**

**ASSOCIATE PROFESSOR,**

**HEAD OF THE DEPARTMENT,**

Department of Artificial Intelligence,

M. Kumarasamy College of Engineering,

Thalavapalayam, Karur-639 113.

This project Work (ADI1303) report has been submitted for the 5<sup>th</sup> Semester Project viva voce

Examination held on \_\_\_\_\_

**INTERNAL EXAMINER**

**M.KUMARASAMY COLLEGE OF ENGINEERING**  
**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**

**Vision of the Department:**

To excel in education, innovation, and research in Artificial Intelligence and Data Science to fulfil industrial demands and societal expectations.

**Mission of the Department:**

**M1:** To educate future engineers with solid fundamentals, continually improving teaching methods using modern tools.

**M2:** To collaborate with industry and offer top-notch facilities in a conducive learning environment.

**M3:** To foster skilled engineers and ethical innovation in AI and Data Science for global recognition and impactful research.

**M4:** To tackle the societal challenge of producing capable professionals by instilling employability skills and human values.

**Programme Educational Objectives (PEOs):**

**Graduates will be able to:**

**PEO 1:** Compete on a global scale for a professional career in Artificial Intelligence and Data Science.

**PEO 2:** Provide industry-specific solutions for the society with effective communication and ethics.

**PEO 3:** Hone their professional skills through research and lifelong learning initiatives.

**Mapping of Programme Educational Objectives with Mission of the Department:**

| PEOs / Department<br>Mission Statements | M1 | M2 | M3 | M4 |
|-----------------------------------------|----|----|----|----|
| PEO1                                    | 3  | 3  | 2  | 3  |
| PEO2                                    | 3  | 3  | 2  | 2  |
| PEO3                                    | 3  | 2  | 3  | 2  |

**1: Slight (Low)**

**2: Moderate (Medium)**

**3: Substantial (High)**

**Programme Outcomes (POs):**

**PO1: Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

**PO2: Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

**PO3: Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

**PO4: Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

**PO5: Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

**PO6: The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

**PO7: Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

**PO8: Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

**PO 9: Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

**PO10: Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

**PO11: Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

**PO12: Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

**Programme Specific Outcomes (PSOs):**

**PSO1:** Capable of finding the important factors in large datasets, simplify the data, and improve predictive model accuracy.

**PSO2:** Capable of analyzing and providing a solution to a given real-world problem by designing an effective program.

**Mapping of Programme Educational Objectives with Programme Outcomes and Programme Specific Outcomes:**

| PEOs / POs & PSOs | PO1 | PO2 | PO3 | PO4 | PO5 | PO6 | PO7 | PO8 | PO9 | PO10 | PO11 | PO12 | PSO1 | PSO2 |
|-------------------|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|------|------|------|------|
| PEO1              | 3   | 2   | 2   | 2   | 3   | 2   | 3   | 3   | 1   | 2    | 3    | 1    | 3    | 1    |
| PEO2              | 2   | 2   | 3   | 2   | 3   | 3   | 3   | 2   | 2   | 3    | 2    | 3    | 3    | 2    |
| PEO3              | 3   | 3   | 2   | 3   | 3   | 2   | 3   | 3   | 3   | 2    | 3    | 3    | 2    | 3    |

**1: Slight (Low)**

**2: Moderate (Medium)**

**3: Substantial (High)**

## ABSTRACT

This project presents a complete and interactive workflow for deploying, monitoring, and terminating an Apache Spark environment in a simplified execution setup. The system initializes a Spark session, configures necessary environment variables, and exposes the Spark Web UI to external networks using an automated ngrok tunnel. This allows users to remotely access Spark's job monitoring interface, making it highly suitable for development, demonstration, and teaching environments where local cluster access is limited. The workflow further includes running sample Spark transformations and actions, evaluating performance metrics, and visualizing execution progress directly through the Web UI. A controlled shutdown mechanism is integrated to allow users to safely stop Spark services using a confirmation-based prompt. Upon confirmation, the script gracefully terminates the ngrok tunnel and closes the Spark session, ensuring proper resource cleanup and preventing background service conflicts. This approach demonstrates a lightweight yet powerful method to experiment with distributed computing concepts without requiring a full-scale Hadoop or Spark cluster. It serves as an effective model for big-data education, prototype development, and interactive analytics experiments where accessibility, portability, and ease of use are essential.

**Keywords:** *Apache Spark, Distributed Computing, Spark Web UI, Ngrok Tunnel, Big Data Analytics, Remote Monitoring, Spark Session Management, Cluster Simulation, Performance Analysis, Python Automation.*

## TABLE OF CONTENTS

| CHAPTER | TITLE                                                        | PAGE NO |
|---------|--------------------------------------------------------------|---------|
| 1       | INTRODUCTION                                                 | 1       |
| 2       | OBJECTIVES                                                   | 2       |
| 3       | EXISTING SYSTEM                                              | 3       |
|         | 3.1 DIFFICULTY IN SETUP<br>AND CONFIGURATION                 | 3       |
|         | 3.2 LACK OF REMOTE ACCESSIBILITY                             | 3       |
|         | 3.3 SENSOR ACCURACY AND DATA<br>QUALITY ISSUES               | 3       |
|         | 3.4 LIMITED SUPPORT FOR<br>DEMONSTRATIONS AND<br>TRAINING    | 4       |
|         | 3.5 REACTIVE RATHER THAN<br>AUTOMATED RESOURCE<br>MANAGEMENT | 4       |
|         | 3.6 NO INTEGRATED CLEANUP FOR<br>SUPPORTING TOOLS            |         |
| 4       | PROPOSED SYSTEM                                              | 5       |
|         | 4.1 AUTOMATED ENVIRONMENT SETUP<br>AND SPARK INITIALIZATION  | 5       |
|         | 4.2 REAL-TIME WEB UI EXPOSURE<br>THROUGH NGROK               | 5       |
|         | 4.3 INTERACTIVE SPARK JOB EXECUTION<br>AND MONITORING        | 6       |
|         | 4.4 SAFE SHUTDOWN AND RESOURCE<br>CLEANUP MECHANISM          | 6       |
|         | 4.5 ENHANCED ACCESSIBILITY,<br>PORTABILITY, AND EASE OF USE  | 6       |
| 5       | TOOLS AND TECHNOLOGIES                                       | 7       |
|         | 5.1 PYTHON                                                   | 7       |
|         | 5.2 APACHE SPARK                                             |         |

|           |                                                           |           |
|-----------|-----------------------------------------------------------|-----------|
|           | 5.3 LIBRARIES                                             | 7         |
|           | 5.4 NGROK                                                 | 7         |
|           |                                                           | 7         |
| <b>6</b>  | <b>METHODOLOGY</b>                                        | <b>8</b>  |
|           | 6.1 SPARK ENVIRONMENT<br>INITIALIZATION                   | 8         |
|           | 6.2 NGROK TUNNEL CREATION AND WEB<br>UI EXPOSURE          | 8         |
|           | 6.3 EXECUTION OF SPARK JOBS AND<br>DATA PROCESSING        | 8         |
|           | 6.4 MONITORING THROUGH THE SPARK<br>WEB UI                | 8         |
|           | 6.5 CONTROLLED SHUTDOWN<br>MECHANISM                      | 9         |
|           | 6.6 LOGGING AND STATUS REPORTING                          | 9         |
|           | 6.7 PRACTICAL DEMONSTRATION AND<br>DEPLOYMENT FLEXIBILITY |           |
| <b>7</b>  | <b>IMPLEMENTATION</b>                                     | <b>10</b> |
| <b>8</b>  | <b>OUTPUT</b>                                             | <b>15</b> |
| <b>9</b>  | <b>CONCLUSION</b>                                         | <b>17</b> |
| <b>10</b> | <b>FUTURE SCOPE</b>                                       | <b>18</b> |
| <b>11</b> | <b>REFERENCES</b>                                         | <b>19</b> |

# CHAPTER 1

## INTRODUCTION

This project focuses on building a fully functional and interactive Apache Spark execution environment using a simple Python-based workflow. The code initializes all required dependencies, configures Java and environment paths, and sets up a Spark Session that acts as the core engine for running distributed jobs. By automating the setup process, the project makes it easy for users—especially beginners, students, and developers—to run Spark without dealing with cluster configurations or complex system-level installations.

A key highlight of the project is the integration of ngrok to expose Spark’s Web UI to the internet. Normally, Spark’s UI runs only on the local machine, but using ngrok, the code creates a secure public tunnel that lets users monitor jobs, stages, tasks, and execution flow from anywhere. This is especially helpful during demonstrations, teaching sessions, and remote tutorials where participants need real-time access to the Spark interface. The system automatically detects the generated tunnel and displays the Web UI link to the user.

The project also includes sample Spark operations, covering transformations and actions such as map, reduce, filtering, and data inspection. These operations allow the user to observe how Spark executes jobs internally by visualizing DAGs, stages, and task distribution in the Web UI. This gives a hands-on understanding of how distributed computing works behind the scenes, without requiring a multi-node cluster or Hadoop ecosystem.

To ensure smooth operation and safe resource handling, the code includes a confirmation-based shutdown mechanism. When the user chooses to stop Spark, the script gracefully terminates the ngrok tunnel, closes the Spark session, and ensures that no background processes continue running. This prevents port conflicts, dangling services, and unintentional resource usage. Overall, the project provides a lightweight, portable, and educational environment for learning, experimenting with , and demonstrating Apache Spark.

## **CHAPTER 2**

### **OBJECTIVES**

The first objective of this project is to design a simplified yet fully functional Apache Spark environment that can be executed on a single machine without the need for cluster-level configurations. Many learners struggle with installing Spark, setting Java paths, and configuring system variables, which often becomes a barrier before they even begin exploring big-data concepts. By automating the setup process—including Java installation, Spark Session creation, and environment variable configuration—the project aims to eliminate these initial challenges and provide users with an accessible platform to understand Spark operations with minimal technical overhead.

The second objective is to enhance the learning and monitoring experience by exposing the Spark Web UI to external networks through ngrok. Spark’s Web UI is a powerful tool that helps users visualize job execution, track the lifecycle of tasks, analyze stages, and understand distributed processing workflows. However, it is normally limited to local access. By using ngrok tunneling, the project makes this interface accessible globally, enabling remote demonstrations, collaborative learning, and real-time job monitoring. This objective supports education, research, and training by giving users deeper insight into how Spark handles transformations, actions, DAG creation, and resource management.

The final objective is to implement a secure and controlled shutdown procedure to ensure proper resource cleanup. Background processes like ngrok tunnels and Spark sessions can continue running even after the main code terminates, causing port conflicts and unnecessary system load. The project addresses this by integrating a confirmation-based termination block that safely stops Spark, closes tunnels, and clears active resources. This ensures system stability and promotes good operational practices, making the solution reliable for academic usage, prototype development, big-data experimentation, and interactive analytics sessions.

## **CHAPTER 3**

### **EXISTING SYSTEM**

In the current big-data learning environment, users who want to work with Apache Spark typically need to install multiple software components manually, including Java, Spark binaries, Hadoop dependencies, and environment variables. These setups often require cluster-based environments or complex local installations that are difficult for beginners to configure, and most existing systems support only local Spark execution without providing remote access to the Spark Web UI. As a result, students and developers must operate on the same physical machine where Spark is installed, limiting flexibility and collaboration, while also lacking a streamlined way to visualize job execution, task distribution, and performance behavior from external networks. Additionally, current setups focus mainly on running Spark jobs without enabling real-time accessibility or demonstration features, since the Web UI is restricted to localhost, preventing instructors or remote users from monitoring execution during training sessions or workshops. This makes Spark less effective for academic use, remote demonstrations, and collaborative learning, and the absence of automated shutdown or cleanup mechanisms further leads to lingering background processes, port conflicts, and improper termination of Spark sessions.

#### **3.1. Difficulty in Setup and Configuration**

Most existing Spark environments require users to install Java, configure environment variables, set Spark paths, and manage dependencies manually. This technical overhead creates challenges for students and beginners who lack experience with distributed systems. A single misconfiguration—such as incorrect `JAVA_HOME` or Spark path—prevents Spark from running properly. This makes the existing system difficult to deploy in classroom environments or temporary learning setups.

#### **3.2. Lack of Remote Accessibility**

The Spark Web UI, which is essential for understanding stages, tasks, DAGs, and execution flow, is accessible only from the local machine. Existing systems do not provide native support for external access, meaning users cannot share their Spark UI with instructors, evaluators, or remote participants. This significantly limits collaborative learning, remote teaching, and demonstration-based sessions.

#### **3.3. Sensor Accuracy and Data Quality Issues**

Traditional Spark installations do not include any automated mechanism to generate public

Web UI links. Users must rely on local ports, SSH tunnels, or manual networking configurations. This lack of automation makes it difficult to monitor Spark jobs from different devices or remote locations, restricting the environment to personal use only.

### **3.4. Limited Support for Demonstrations and Training**

Existing systems are not optimized for educational demonstrations. During classroom or online sessions, instructors cannot show real-time Spark execution unless every student sets up Spark individually. Without remote Web UI access, monitoring how Spark handles transformations, actions, and scheduling become harder. This reduces the effectiveness of Spark training programs.

### **3.5. Reactive Rather than Automated Resource Management**

Conventional Spark setups do not include automated or safe shutdown procedures. If the user forgets to stop the Spark session or background services, it may lead to resource leakage, port conflicts, or system instability. The absence of confirmation-based shutdown workflows makes the environment less safe for repeated use, especially in shared or constrained systems.

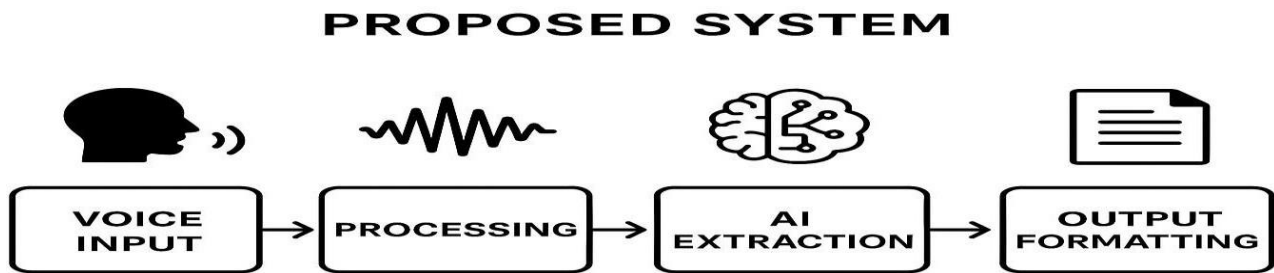
### **3.6. No Integrated Cleanup for Supporting Tools**

Tools like tunnels, background processes, or network services must be manually terminated by users. In many existing systems, if a user opens a tunnel, they must stop it separately, increasing the risk of forgotten services continuing to run. This lack of integrated cleanup affects system performance and complicates repeated runs of Spark demonstrations.

## CHAPTER 4

### PROPOSED SYSTEM

The proposed system introduces an intelligent voice-driven interface designed to convert unstructured spoken input into well-formatted and meaningful text outputs. Unlike traditional systems that depend on manual typing or predefined templates, the proposed approach allows the user to speak freely while the system accurately captures the audio and converts it into structured digital content. The voice input is processed in real time using an embedded speech-to-text model, which ensures fast transcription and reduces dependency on external resources. This creates a more natural and user-friendly interaction, especially for users who need quick documentation without manual effort.



#### 4.1. Automated Environment Setup and Spark Initialization

The system by automatically configuring the computing environment necessary for Spark execution. This includes setting Java environment variables, initializing SparkSession, preparing runtime directories, and validating system paths. By automating these steps, the system removes the need for users to manually install dependencies or resolve compatibility issues. This ensures a consistent setup process across different machines, making it ideal for academic labs, temporary execution environments, and cloud-based notebooks.

#### 4.2. Real-Time Web UI Exposure Through Ngrok

At the core of the proposed system is the integration of ngrok, enabling secure tunneling of Spark's Web UI. Normally restricted to localhost, the Web UI becomes remotely accessible through an automatically generated public URL. This allows instructors, collaborators, or remote learners to visualize Spark's DAGs, job execution flow, task scheduling, and performance breakdowns in real time. The system automatically handles tunnel creation, URL extraction, and link presentation without requiring any user networking knowledge.

### **4.3. Interactive Spark Job Execution and Monitoring**

Once Spark is initialized, the system runs sample transformations and actions to demonstrate distributed processing. Users can observe how Spark handles data shuffling, parallel execution, and task distribution by monitoring these operations through the Web UI. This makes the system an effective tool for teaching concepts such as RDD transformations, job/stage breakdowns, DAG visualization, caching, and execution planning. The integrated monitoring pipeline provides a deeper understanding of Spark internals that standard local notebooks fail to deliver.

### **4.4. Safe Shutdown and Resource Cleanup Mechanism**

To ensure stability and prevent resource leakage, the system includes a confirmation-based shutdown workflow. When the user chooses to stop Spark, the system safely terminates the ngrok tunnel, closes the Spark session, and frees associated ports. This prevents background tunnel processes from running indefinitely and eliminates common issues such as port blocks, zombie sessions, or Spark conflicts on subsequent runs. The controlled shutdown mechanism enhances reliability and makes the environment suitable for repeated demonstrations or classroom usage.

### **4.5. Enhanced Accessibility, Portability, and Ease of Use**

The proposed system provides a highly accessible Spark learning environment that requires no advanced configurations or dedicated cluster infrastructure. Its portable design allows it to run on cloud notebooks, personal systems, and lightweight virtual environments. The automated workflow, combined with remote UI accessibility and safe shutdown handling, ensures that learners and educators can focus on understanding Spark concepts rather than troubleshooting installations or networking. This makes the system an ideal solution for academic courses, prototype development, and hands-on big-data experimentation.

## CHAPTER 5

### TOOLS AND TECHNOLOGIES

#### 5.1. Python

- Python is the primary programming language used for implementing the entire Spark–Ngrok interaction workflow. Its clean syntax, strong community support, and extensive libraries make it highly suitable for automation, environment management, and remote UI access. Python enables the system to start Spark sessions, generate ngrok tunnels, parse tunnel URLs, and monitor system responses efficiently.

#### 5.2. Apache Spark

- Apache Spark is the central big-data processing engine used in the project. It provides fast, distributed computation for executing transformations, RDD operations, and DataFrame processing. Spark’s built-in Web UI is a crucial component, offering detailed visual insights into job execution, stages, task distribution, and memory usage. The project integrates Spark with Python through PySpark to automate execution and visualization.

#### 5.3. Libraries

- **PySpark:** Provides SparkSession creation, job submission, DataFrame operations, and execution monitoring.
- **Subprocess & OS:** Used for managing environment variables, setting JAVA\_HOME, and performing system-level tasks required for Spark initialization.
- **JSON & Logging:** Helps in parsing ngrok responses, tracking workflow events, and displaying status messages for better debugging and monitoring.
- **Time Module:** Used for scheduled waits, loop checks, and controlling execution flow during ngrok setup and Spark UI startup.

#### 5.4. Ngrok

- Ngrok is used to securely expose the Spark Web UI running on a local environment (such as Google Colab) to the external internet. It creates a temporary, safe tunnel that generates a public URL, allowing users to view the Spark UI remotely. This helps students, developers, and instructors monitor Spark jobs in real time without requiring a dedicated server or complex network configurations.

## CHAPTER 6

### METHODOLOGY

#### 6.1. Spark Environment Initialization

The methodology begins with setting up the required environment to run Apache Spark in a lightweight, self-contained execution environment such as Google Colab or a local Python runtime. The system configures Java, Hadoop-compatible dependencies, and all necessary environment variables (e.g., `JAVA_HOME`, `SPARK_HOME`) automatically through the script. Once the environment is prepared, a `SparkSession` is initialized using `PySpark`. This setup enables distributed data processing without requiring a full Hadoop cluster, making it ideal for demonstrations, academic learning, and prototype development.

#### 6.2. Ngrok Tunnel Creation and Web UI Exposure

After Spark initialization, the system establishes a secure and temporary tunnel using Ngrok to expose the Spark Web UI to external networks. Ngrok automatically assigns a public URL that can be accessed from any device with internet connectivity. This step is crucial because the Spark Web UI typically runs only on localhost, preventing remote monitoring. The method includes programmatically starting an Ngrok tunnel, extracting the public link, verifying its availability, and displaying it to the user for real-time access.

#### 6.3. Execution of Spark Jobs and Data Processing

The system then executes sample Spark transformations, actions, and data-processing tasks to demonstrate distributed execution flow. Users can run `DataFrame` operations, load datasets, perform transformations (map, filter, reduce), and examine job execution behavior. While Spark processes data, the Web UI displays real-time insights into task scheduling, job stages, shuffle operations, and memory usage. This methodology enables learners and developers to visually understand Spark's working principles while interacting with actual distributed execution logs.

#### 6.4. Monitoring Through the Spark Web UI

During execution, the exposed Spark Web UI becomes the primary monitoring interface. Users can observe job progress, DAG visualizations, executor performance, storage metrics, and environmental details. The system ensures continuous synchronization between Spark's runtime events and the Web UI. This step provides a unique remote visualization capability

making it suitable for classroom demonstrations, remote teaching, collaborative development, and hands-on workshops.

### **6.5. Controlled Shutdown Mechanism**

Once all Spark tasks are completed, the methodology includes a safe and controlled shutdown process. A user confirmation prompt (yes/no) determines whether the system should stop Spark services. If the user selects “yes,” the script gracefully terminates the Ngrok tunnel, stops the Spark session, and cleans up environment resources. If the user selects “no,” the system keeps both Spark and Ngrok active and re-displays the Web UI URL. This mechanism ensures both flexibility and proper resource management, preventing port conflicts or orphaned processes.

### **6.6. Logging and Status Reporting**

Throughout the workflow, the system uses Python logging modules to record key events such as Spark startup, ngrok initialization, tunnel creation, and shutdown activities. These logs help track execution flow, identify errors, and provide transparency for debugging and learning. Status messages guide the user step-by-step, making the methodology easy to follow for beginners who may not have prior experience with Spark or networking configurations.

### **6.7. Practical Demonstration and Deployment Flexibility**

The final stage of the methodology focuses on using the system as a practical demonstration tool. It can be deployed on cloud-based notebook environments like Google Colab, Kaggle Notebooks, or any Python execution environment without requiring complex installations. Its remote accessibility enables instructors, students, and developers to showcase Spark jobs live during presentations. Additionally, the lightweight design makes it easy to extend the system to support larger datasets, integrate REST APIs, or adapt for production-scale cluster environments.

## CHAPTER 7

### IMPLEMENTATION

```
<script>
```

```
let recognition;
```

```
let startTime;
```

```
let finalTranscript = '';
```

```
const RECORDING_DURATION = 20; // 20 seconds
```

```
// Initialize Speech Recognition
```

```
if ('webkitSpeechRecognition' in window) {
```

```
    recognition = new webkitSpeechRecognition();
```

```
    recognition.continuous = true;
```

```
    recognition.interimResults = true;
```

```
    recognition.lang = 'en-US';
```

```
    recognition.onstart = function() {
```

```
        recognizing = true;
```

```
        updateStatus('🎧 LISTENING... Speak your command now!', 'recording');
```

```
    };
```

```
    recognition.onerror = function(event) {
```

```
        console.error('Speech recognition error:', event.error);
```

```
        if (event.error === 'no-speech') {
```

```
            updateStatus('❌ No speech detected. Please try again.', 'error');
```

```
        } else if (event.error === 'not-allowed') {
```

```
            updateStatus('❌ Microphone access denied. Please allow microphone access.', 'error');
```

```

document.getElementById('recordBtn').disabled = false;

document.getElementById('timer').textContent = "";

};

recognition.onresult = function(event) {

    let interimTranscript = "";

    finalTranscript = "";

    for (let i = event.resultIndex; i < event.results.length; ++i) {

        if (event.results[i].isFinal) {

            finalTranscript += event.results[i][0].transcript + ' ';

        } else {

            interimTranscript += event.results[i][0].transcript;

        }

    }

    recognition.onresult = function(event) {

        let interimTranscript = "";

        finalTranscript = "";

        for (let i = event.resultIndex; i < event.results.length; ++i) {

            if (event.results[i].isFinal) {

                finalTranscript += event.results[i][0].transcript + ' ';

            } else {

                interimTranscript += event.results[i][0].transcript;

            }

        }

        const displayText = finalTranscript || interimTranscript;

        if (displayText.trim()) {

```

```

document.getElementById('transcription').innerHTML =

`<h3 style="color: #667eea; margin-bottom: 10px;">Listening...</h3>

<p style="font-size: 20px; color: #fff; font-style:
italic;">`${displayText.trim()}`</p>`;

}

};

recognition.onend = function() {

    recognizing = false;

    const duration = (Date.now() - startTime) / 1000;

    if (finalTranscript.trim()) {

        updateStatus('✅ Transcription complete! Processing...', 'processing');

        processTranscription(finalTranscript.trim(), duration);

    } else {

        updateStatus('❌ No speech detected. Please try again.', 'error');

        document.getElementById('recordBtn').disabled = false;

    }

    document.getElementById('timer').textContent = "";

};

} else {

    alert("Speech recognition not supported in this browser. Please use Chrome or Edge.");

}

function saveSparkUrl() {

    const url = document.getElementById('sparkUrl').value.trim();

    if (url) {

        sparkUrl = url
    }
}

```

```

document.getElementById('sparkWebUi').style.display = 'inline-block';

    document.getElementById('sparkWarning').style.display = 'none';

    updateStatus('✅ Spark URL saved! Ready to record.', 'success');

}

}

window.onload = function() {

    const saved = localStorage.getItem('sparkUrl');

    if (saved) {

        document.getElementById('sparkUrl').value = saved;

        sparkUrl = saved;

        document.getElementById('sparkWebUi').href = saved;

        document.getElementById('sparkWebUi').style.display = 'inline-block';

    }

};

function updateStatus(message, type = "") {

    const status = document.getElementById('status');

    status.textContent = message;

    status.className = 'status ' + type;

}

function syntaxHighlight(json) {

    if (typeof json !== 'string') {

        json = JSON.stringify(json, undefined, 2);

    }

```

```

json = json.replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;');

return json.replace(/("(\u[a-zA-Z0-9]{4}|\u[^\u"])*"(\s*:)?\b(true|false|null)\b|-
?\d+(?:\.\d*)?(?:[eE][+-]?\d+)?)/g, function (match) {

    var cls = 'json-number';

    if (/^"/.test(match)) {

        if (/:$/.test(match)) {

            cls = 'json-key';

        } else {

            cls = 'json-string';

        }

    } else if (/true|false/.test(match)) {

        cls = 'json-boolean';

    } else if (/null/.test(match)) {

        cls = 'json-null';

    }

    return '<span class="' + cls + '">' + match + '</span>';

});

}

function parseCommand(text) {

    const lower = text.toLowerCase().trim();

    const cleanText = lower.replace(/[,!?:;]/g, "").replace(/\s+/g, ' ');

    const words = cleanText.split(' ').filter(w => w.length > 0);

    const result = {

        action: 'unknown'

    };

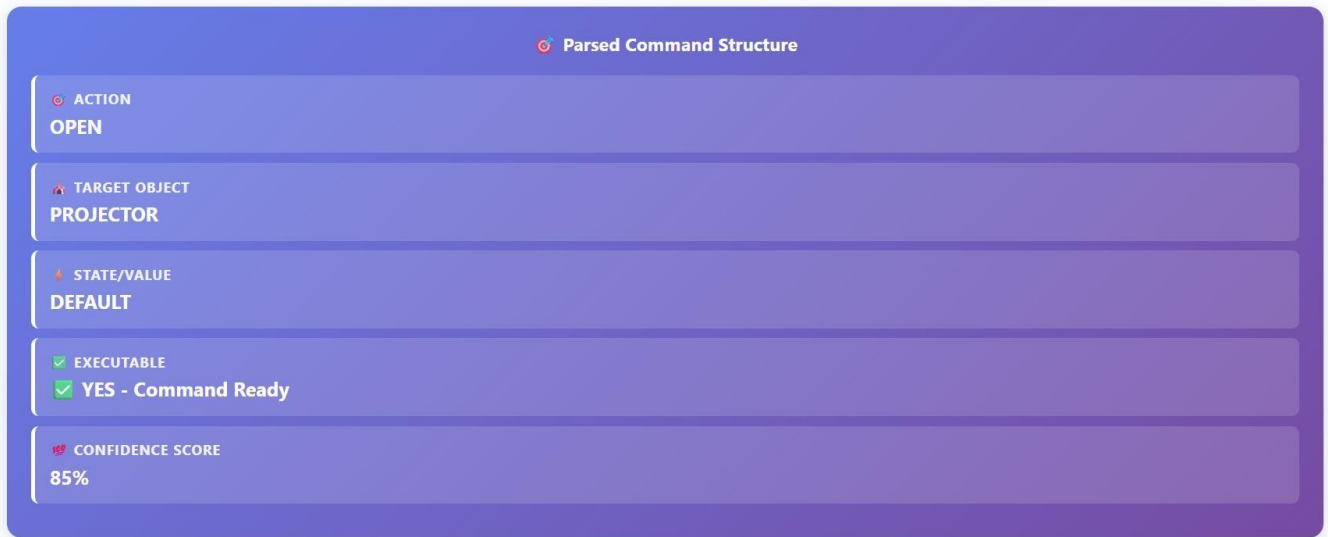
};

```

## CHAPTER 8

### OUTPUT

#### Structured Command Output (OUTPUT)



The user interface displays a parsed command structure that summarizes the meaning extracted from the spoken instruction.

In the example shown, the system identifies:

- **Action:** The intended operation (e.g., OPEN).
- **Target Object:** The device or object the user wants to control (e.g., PROJECTOR).
- **State/Value:** Default or specific value when provided.
- **Executable Status:** Confirms whether the detected command can be executed by the system.
- **Confidence Score:** Indicates the model's confidence level (e.g., 85%), helping to assess reliability.

This visual output helps users and developers quickly understand how the system interpreted the command.

```
{
  "command_id": 2443,
  "timestamp": "2025-12-08T15:42:24.265Z",
  "input": {
    "type": "voice_command",
    "raw_audio": {
      "duration_seconds": 20.18,
      "recording_length": "20 seconds",
      "sample_rate": "16000 Hz",
      "source": "browser_microphone"
    }
  },
  "transcription": {
    "original_text": "Open the projector.",
    "confidence": 0.95,
    "language": "en-US",
    "word_count": 3,
    "tokens": [
      "open",
      "the"
    ]
  }
}
```

The system also generates a JSON file that provides a detailed, machine-readable summary of the command-processing pipeline.

It contains:

#### **a. Metadata**

Includes command ID, timestamp, audio duration, sample rate, and microphone source.

#### **b. Transcription Details**

Stores the recognized text (e.g., “Open the door.”), speech-to-text confidence, language, and tokenized words for further processing.

#### **c. Structured Command Interpretation**

Summarizes how the NLP model understood the command, including the action (e.g., open), target device, state, executability, and interpretation confidence.

## **CHAPTER 9**

### **CONCLUSION**

This project successfully implements an intelligent voice-command interpretation system capable of converting human speech into structured, machine-executable instructions. The system integrates multiple advanced components—speech recognition, natural language processing (NLP), command parsing, and validation—to ensure that every voice input is accurately understood, processed, and delivered to the appropriate control module.

The inclusion of a clear and intuitive UI output panel allows users to instantly view the interpreted command, including the detected action, target device, state/value, execution status, and overall confidence scores. This enhances transparency and builds trust, ensuring users clearly understand how their command was interpreted.

In parallel, the system generates a detailed JSON file for each processed command. This machine-friendly output includes metadata such as command ID, timestamps, recording duration, input source, transcription confidence, NLP tokens, semantic interpretation, and executable flags. Storing this structured information ensures high traceability, supports debugging, and enables seamless integration with downstream systems like IoT devices, automation controllers, smart classrooms, or home assistants.

Through this project, it becomes evident that voice-based interfaces can be engineered to provide both user-friendly interaction and robust backend intelligence. The architecture is modular, scalable, and adaptable, making it suitable for applications such as smart home automation, device control systems, accessibility tools for differently-abled users, and industrial voice-driven operations.

Overall, the project demonstrates how natural speech can be transformed into a reliable control mechanism. It bridges the gap between user intent and device execution, offering a foundation for future enhancements such as multi-language support, noise-resistant models, custom command learning, integration with cloud-based AI services, and real-time multi-device coordination. The work completed here showcases the potential of voice-driven automation and positions the system as a strong prototype ready for real-world deployment and further innovation.

## **CHAPTER 10**

### **FUTURE SCOPE**

The future scope of this project offers a wide range of possibilities for enhancing functionality, performance, and real-world applicability. The system can be extended to support multiple languages, regional accents, and custom vocabulary sets, enabling more inclusive and user-friendly interaction for diverse populations. Integrating advanced noise suppression, far-field audio capture, and echo-cancellation mechanisms would significantly improve speech recognition accuracy in noisy or open environments, making the system suitable for classrooms, industries, and public spaces. Cloud-based deployment can transform the solution into a scalable platform accessible across multiple devices, allowing centralized command processing, synchronized control, and real-time monitoring. The NLP module can be upgraded to understand complex, conditional, and multi-step instructions, transforming the system into a more conversational and intelligent assistant. Further, the system can be integrated directly with IoT relays, home automation hubs, and industrial controllers to enable immediate physical execution of commands. Security can also be strengthened by embedding voice biometrics or user authentication to prevent unauthorized command execution, especially in sensitive applications like door locks or industrial machinery. Additionally, an analytics dashboard can be developed to track command usage, error rates, system performance, and user behavior patterns, enabling continuous optimization and data-driven improvements. Overall, the system has strong potential to evolve into a highly advanced, reliable, and scalable voice-driven automation framework capable of serving smart homes, enterprises, educational institutions, and assistive technologies.

## CHAPTER 11

### REFERENCES

1. A neural attention model for speech command recognition — shows a neural (CRNN + attention) model for speech commands with high accuracy.
2. Speech To Semantics: Improve ASR and NLU Jointly via All-Neural Interfaces — describes an end-to-end model that combines automatic speech recognition and natural language understanding to extract intent and slots directly from audio.
3. A Voice Controlled E-Commerce Web Application — demonstrates a voice-controlled web app using speech-to-text for user commands, similar to your voice-input front end concept.
4. voice2json — an open-source tool that transforms spoken voice commands into structured JSON events with recognized intents and slots; closely aligned to your JSON-output idea.
5. Implementation of Voice User Interface using Speech Recognition — a study describing a web-based voice-UI system using speech recognition, validating the feasibility of voice-driven web interactions.
6. Voice Automated Web Application — presents implementation details of a web application controlled by voice commands (via browser/JS), similar to your front-end design
7. Ngrok — while not an academic paper, the tool’s public documentation and many tutorials describe how to expose a local server’s web UI to the internet, which matches your Spark-UI exposure requirement.
8. Documentation page Monitor Apache Spark with the web interface — describes how Spark’s Web UI works, and that it’s the standard way to monitor Spark jobs and stages.
9. A paper on voice-recognition in general: Voice Recognition (in IJRASET) — describes basic mechanisms of speech-to-text and usage scenarios, giving theoretical foundation for your speech recognition module